

Namespace CommonNovel

Classes

[Compiler](#)

1. Introduction

1.1. What is Anov?

Anov (Advanced-Novel) is a plain text format for creating novel games, based on conventions for indicating formatting in the novel game of Alice Novel. It was developed by [Lemon73](#) and distributed in 2023 in the form of the [Alice Novel v0.9.0-alpha](#) for running novel games.

1.2. Why is a spec needed?

Alice Novel v0.9.0-alpha to v0.9.4 do not specify the syntax unambiguously. (Ignore [Alice Docs](#))

To make matters worse, because nothing in Anov counts as a “syntax error,” the divergence often isn’t discovered right away.

1.3. About this document

This document attempts to specify Anov syntax unambiguously. It contains many examples with side-by-side Anov and NovellL.

In the examples, the `→` character is used to represent tabs.

2. Preliminaries

2.1. Characters and lines

To Be Determined...

2.2. Tabs

To Be Determined...

2.3. Insecure characters

To Be Determined...

2.4. Backslash escapes

To Be Determined...

3. Nodes and lines

3.1. What is a node?

The CommonNovel uses divisions called **nodes**. These are the smallest units of reading in novel games, and each reading is performed on a node-by-node basis.

A node indicates multiple lines from a blank line (a line containing only spaces, tabs, or line feed codes) to the next blank line.

The node indicates an interval of zero or more lines, with (theoretically) no upper limit.

Example 1

```
- Alice
[Hi!]

- Rabbit
[Hi, there.]
```

The first node includes the two lines; - Alice and [Hi!]. The next node includes the two lines; - Rabbit and [Hi, there.].

Example 2

```
- Alice
[Hi!]

- Rabbit
[Hi, there.]
```

This example is similar to [Example 1](#), however there are two blank lines between [Hi!] and - Rabbit. So, the first node includes the two lines; - Alice and [Hi!]. But the next node does not include any line. (Treat as blank internally) Then, the last node includes the two lines; - Rabbit and [Hi, there.].

Example 3

```
- Alice
[Good morning!]
[Hi.]
```

In this example, the same command is used twice in one node. In this case, the latter, [Hi.], takes precedence.

This is not an error, but it is not recommended. It is recommended that implementations show a warning, for example: `[Duplicate] The former is not used and should be deleted.`

4. Basic Commands

This section describes the different kinds of commands that make up an Anov document. Extended commands added by plug-ins may be available, but this section describes the basic commands that can be used without plug-ins.

4.1. Chat

(Before conversion: `main.anov`)

```
[text]
```

(After conversion: `main.nil`)

```
{
  "cells": [
    {
      "messages": "text" // or "text"
    }
  ]
}
```

4.2. Character name

(Before conversion: `main.anov`)

```
- character-name
```

(After conversion: `main.nil`)

```
{
  "cells": [
    {
      "character-name": "character-name", // or "name"
      "images": {
        "foreground": "character-name.png" // or "character"
      }
    }
  ]
}
```

```
]
}
```

Also, this method can be used with [the emotion command](#). See that section for more details.

4.3. Character image / Emotion

(Before conversion: `main.anov`)

```
/ emotion
```

(After conversion: `main.nil`)

```
{
  "cells": [
    {
      "images": {
        "foreground": "(character-name)-emotion.png" // or "character"
      }
    }
  ]
}
```

Basically, this method is used with [the character command](#).

(Before conversion: `main.anov`)

```
- character-name / emotion
```

(After conversion: `main.nil`)

```
{
  "cells": [
    {
      "character-name": "character-name", // or "name"
      "images": {
        "foreground": "character-name-emotion.png" // or "character"
      }
    }
  ]
}
```

4.4. Background image / Location

(Before conversion: `main.anov`)

```
> location
```

(After conversion: `main.nil`)

```
{
  "cells": [
    {
      "images": {
        "background": "location.png"
      }
    }
  ]
}
```

4.5. Play audio

- `background`: The user can proceed to the next node while the audio is playing.
- `foreground`: The user cannot proceed to the next node until the audio finishes playing.

(Before conversion: `main.anov`)

```
audio: music
```

(After conversion: `main.nil`)

```
{
  "cells": [
    {
      "musics": { // or "audio"
        "background": "music.mp3"
      }
    }
  ]
}
```

Stopping audio

(Before conversion: `main.anov`)

audio:

(After conversion: `main.nil`)

```
{
  "cells": [
    {
      "musics": { // or "audio"
        "background": ""
      }
    }
  ]
}
```

4.6. Play movie

- `background`: The user can proceed to the next node while the movie is playing.
- `foreground`: The user cannot proceed to the next node until the movie finishes playing.

(Before conversion: `main.anov`)

movie: movie

(After conversion: `main.nil`)

```
{
  "cells": [
    {
      "movies": {
        "foreground": "movie.mp4"
      }
    }
  ]
}
```

5. Under discussion

This section describes new functions that are under discussion.

5.1. Extension linking to external files

When loading external files such as image files, the file extension must be specified.

Currently, the location command `> location` serves both to specify the location and display the image, but there is debate as to whether it should be `> location.png`, including the file extension. At present, `location.png`, including the file extension, is prioritised.

5.2. Flags/Branching

In order to perform branch processing such as scenario branching, it is necessary to add flags and conditional branching specifications.

5.3. Arguments

There are situations where arguments are required, such as specifying the type of animation or the number of seconds. We are currently considering syntax such as `:fading` or `{fading}`.

5.4. Plug-in system

(Advanced) It is necessary to consider introducing a plug-in system to enable processing other than basic commands. Plug-ins must be implemented with security issues in mind.

5.5. Linking to another file

Example:

```
# main.anov
<next.anov>
```

This example shows linking from `main.anov` to `next.anov`.

5.6. Anproj-format

Anproj-format is a ZIP-based file format that includes Anov files, config files, image files, and other files. CommonNovel does not have an alternative.

Appendix

License

[CC-BY-SA 4.0](#)

Copyright © 2023-2025, Alice Project.

References

- [commonmark-spec 0.31.2](#) 